



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
INGENIERÍA EN COMPUTACIÓN

Sistemas Distribuids

Proyecto 3. Servicio de operaciones sobre archivos remotos mediante gRPC

Nombre de los integrantes:

- Ayala Hernández María Fernanda
- Saldaña Navarrete Andrea
- Ugartechea González Luis Antonio
- Tepal Briseño Hansel Yael

NOMBRE DEL PROFESOR: ELBA KAREN SÁENZ GARCÍA
GRUPO: 03
SEMESTRE: 2027-1

FECHA DE ENTREGA: 9 de noviembre del 2026

Objetivo

Diseñar e implementar un servicio de archivos remotos mediante RPC, definiendo procedimientos y mensajes con Protocol Buffers, validando las solicitudes y gestionando de forma segura los errores relacionados con archivos.

Introducción

En este segundo proyecto se diseña e implementa un servicio remoto de gestión de archivos utilizando gRPC. A diferencia del proyecto anterior enfocado en difusión (uno a muchos), este sistema implementa una arquitectura cliente-servidor clásica (uno a uno), en donde el cliente no accede directamente al sistema de archivos local, sino que delega todas las operaciones al servidor remoto a través de llamadas a procedimientos remotos (RPC).

El sistema soporta cinco operaciones fundamentales: crear un archivo nuevo, leer su contenido, agregar texto (sin sobrescribir), listar el directorio y eliminar un archivo. El núcleo de este desarrollo se centra en la **validación y seguridad**, garantizando que todas las solicitudes operen estrictamente dentro de un directorio de trabajo confinado y rechazando enérgicamente cualquier intento de escape o escalamiento de privilegios (Path Traversal).

Desarrollo e Implementación

1. Preparación del Entorno

Para aislar las dependencias del proyecto del sistema operativo host, se configuró un entorno virtual de Python. Los comandos ejecutados para inicializar el ambiente fueron:

```
python3 -m venv .venv
source .venv/bin/activate
```

Acto seguido, se instalaron las librerías oficiales de gRPC necesarias para compilar el protocolo y ejecutar los nodos:

```
python3 -m pip install --upgrade pip
python3 -m pip install grpcio grpcio-tools
```

2. Definición del Protocolo y Preparación de Archivos

El archivo `archivos.proto` define las interfaces de comunicación. Para facilitar un manejo de errores claro (como lo exige el proyecto), todos los mensajes de respuesta (`Response`) comparten una estructura base: un booleano `exito` para indicar el estado de la operación y un `mensaje` de texto con detalles legibles para el usuario.

```
syntax = "proto3";
package gestor_archivos;

message CrearRequest { string nombre_archivo = 1; string contenido = 2; }
message CrearResponse { bool exito = 1; string mensaje = 2; }

message LeerRequest { string nombre_archivo = 1; }
message LeerResponse { bool exito = 1; string mensaje = 2; string contenido = 3; }

message AgregarRequest { string nombre_archivo = 1; string contenido = 2; }
message AgregarResponse { bool exito = 1; string mensaje = 2; }

message ListarRequest {}
message ListarResponse { bool exito = 1; string mensaje = 2; repeated string archivos = 3; }

message EliminarRequest { string nombre_archivo = 1; }
message EliminarResponse { bool exito = 1; string mensaje = 2; }

service ServicioArchivos {
    rpc CrearArchivo (CrearRequest) returns (CrearResponse);
    rpc LeerArchivo (LeerRequest) returns (LeerResponse);
    rpc AgregarContenido (AgregarRequest) returns (AgregarResponse);
    rpc ListarArchivos (ListarRequest) returns (ListarResponse);
    rpc EliminarArchivo (EliminarRequest) returns (EliminarResponse)
       ;
}
```

Los archivos base de Python (`archivos_pb2.py` y `archivos_pb2_grpc.py`) se generaron mediante el compilador `protoc`:

```
python3 -m grpc_tools.protoc -I. --python_out=. --grpc_python_out=. archivos.proto
```

3. Diagrama de Arquitectura

El diseño sigue una arquitectura centralizada y confinada. El servidor administra de forma exclusiva un directorio (ej. `almacen_remoto/`). El cliente, por su parte, es meramente una interfaz interactiva; recolecta las entradas del usuario y dispara las solicitudes. El servidor procesa, valida, toca el disco duro si es seguro, y retorna el estado.

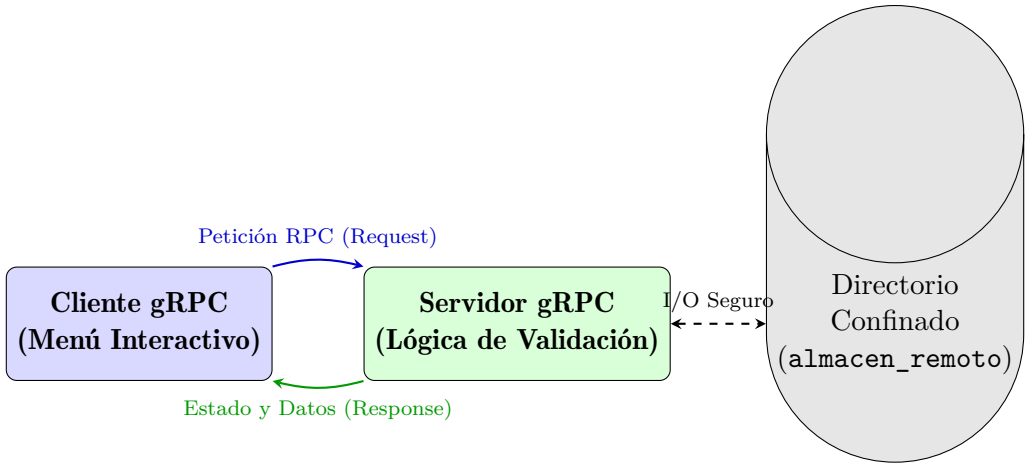


Figure 1: Arquitectura del Servicio de Archivos Remotos.

4. Lógica del Servidor (`servidor.py`)

La clase `ServicioArchivos` implementa los métodos RPC definidos en `archivos.proto`. Cada método recibe una solicitud, valida los datos necesarios y devuelve un mensaje de respuesta con el resultado. Las operaciones sobre nombres de archivo pasan primero por `_validar_seguridad`.

Inicialización: `__init__()`

El constructor comprueba que exista el directorio de almacenamiento. Si todavía no existe, lo crea. Esto prepara la carpeta antes de atender las solicitudes, sin borrar archivos que ya estén guardados.

```
def __init__(self):
    if not os.path.exists(DIRECTORIO_ALMACEN):
        os.makedirs(DIRECTORIO_ALMACEN)
        print(f"Carpeta '{DIRECTORIO_ALMACEN}' creada.")
```

Validación de nombres: `_validar_seguridad()`

Este método rechaza nombres que contienen `..`, `/` o
. Así impide que una solicitud incluya rutas relativas o subdirectorios. Si el nombre pasa las comprobaciones, combina el nombre con `DIRECTORIO_ALMACEN`; en caso contrario devuelve `None`. La ruta construida es relativa al directorio de trabajo del proceso.

```
def _validar_seguridad(self, nombre_archivo):
    if ".." in nombre_archivo or "/" in nombre_archivo or "\\" in
       nombre_archivo:
        return None
```

```
    return os.path.join(DIRECTORIO_ALMACEN, nombre_archivo)
```

Crear archivo: `CrearArchivo()`

Primero valida el nombre. Después comprueba si el archivo ya existe para evitar sobrescribirlo. Si no existe, lo crea en modo `w`, escribe el contenido recibido en UTF-8 y responde con `exito=True`. Los errores de escritura se convierten en una respuesta fallida con el detalle de la excepción.

```
def CrearArchivo(self, request, context):
    ruta_segura = self._validar_seguridad(request.nombre_archivo)
    if not ruta_segura:
        return archivos_pb2.CrearResponse(exito=False,
            mensaje="Error de seguridad: Nombre de archivo inválido.
               ")
    if os.path.exists(ruta_segura):
        return archivos_pb2.CrearResponse(exito=False,
            mensaje=f"Error: El archivo '{request.nombre_archivo}'
               ya existe.")
    try:
        with open(ruta_segura, 'w', encoding='utf-8') as archivo:
            archivo.write(request.contenido)
        return archivos_pb2.CrearResponse(exito=True,
            mensaje=f"Archivo '{request.nombre_archivo}' creado
               exitosamente.")
    except Exception as error:
        return archivos_pb2.CrearResponse(exito=False, mensaje=str(
           error))
```

Leer archivo: `LeerArchivo()`

Valida el nombre y confirma que el archivo exista antes de abrirlo en modo lectura. Si la lectura termina correctamente, devuelve tanto el estado como el contenido; si falla la validación, no existe el archivo o ocurre un error de entrada/salida, devuelve `exito=False` y contenido vacío.

```
def LeerArchivo(self, request, context):
    ruta_segura = self._validar_seguridad(request.nombre_archivo)
    if not ruta_segura:
        return archivos_pb2.LeerResponse(exito=False,
            mensaje="Error de seguridad: Nombre de archivo inválido.
               ", contenido="")
    if not os.path.exists(ruta_segura):
        return archivos_pb2.LeerResponse(exito=False,
            mensaje=f"Error: El archivo '{request.nombre_archivo}'
               no existe.",
```

```
            contenido="")
    try:
        with open(ruta_segura, 'r', encoding='utf-8') as archivo:
            contenido = archivo.read()
        return archivos_pb2.LeerResponse(exito=True,
            mensaje="Lectura exitosa.", contenido=contenido)
    except Exception as error:
        return archivos_pb2.LeerResponse(exito=False,
            mensaje=str(error), contenido="")
```

Agregar contenido: `AgregarContenido()`

Comprueba que el nombre sea válido y que el archivo ya exista. Lo abre con el modo `a` (*append*), que añade los datos al final sin reemplazar el contenido anterior. El método del cliente agrega un salto de línea antes del texto ingresado.

```
def AgregarContenido(self, request, context):
    ruta_segura = self._validar_seguridad(request.nombre_archivo)
    if not ruta_segura:
        return archivos_pb2.AgregarResponse(exito=False,
            mensaje="Error de seguridad: Nombre de archivo inválido.
               ")
    if not os.path.exists(ruta_segura):
        return archivos_pb2.AgregarResponse(exito=False,
            mensaje=f"Error: El archivo '{request.nombre_archivo}'
               no existe.")
    try:
        with open(ruta_segura, 'a', encoding='utf-8') as archivo:
            archivo.write(request.contenido)
        return archivos_pb2.AgregarResponse(exito=True,
            mensaje=f"Contenido agregado exitosamente a '{request.
               nombre_archivo}'.")
    except Exception as error:
        return archivos_pb2.AgregarResponse(exito=False, mensaje=str
           (error))
```

Listar archivos: `ListarArchivos()`

No requiere un nombre de archivo en la solicitud. Obtiene las entradas del directorio con `os.listdir` y las devuelve en el campo repetido `archivos`. Si ocurre un error, responde con una lista vacía y el mensaje correspondiente.

```
def ListarArchivos(self, request, context):
    try:
        archivos = os.listdir(DIRECTORIO_ALMACEN)
        return archivos_pb2.ListarResponse(exito=True,
```

```
            mensaje="Listado generado exitosamente.", archivos=
                archivos)
    except Exception as error:
        return archivos_pb2.ListarResponse(exito=False,
            mensaje=str(error), archivos=[])
```

Eliminar archivo: `EliminarArchivo()`

Valida el nombre, verifica que el archivo exista y lo elimina con `os.remove`. Cada condición de error produce una respuesta fallida; una eliminación correcta se confirma mediante el campo `exito`.

```
def EliminarArchivo(self, request, context):
    ruta_segura = self._validar_seguridad(request.nombre_archivo)
    if not ruta_segura:
        return archivos_pb2.EliminarResponse(exito=False,
            mensaje="Error de seguridad: Nombre de archivo inválido.
                ")
    if not os.path.exists(ruta_segura):
        return archivos_pb2.EliminarResponse(exito=False,
            mensaje=f"Error: El archivo '{request.nombre_archivo}'
                no existe.")
    try:
        os.remove(ruta_segura)
        return archivos_pb2.EliminarResponse(exito=True,
            mensaje=f"Archivo '{request.nombre_archivo}' eliminado
                exitosamente.")
    except Exception as error:
        return archivos_pb2.EliminarResponse(exito=False, mensaje=
            str(error))
```

Arranque del servidor: `iniciar_servidor()`

Esta función crea el servidor gRPC con un grupo de hasta diez hilos, instancia el servicio y registra sus métodos RPC. Después enlaza el servicio al puerto indicado en todas las interfaces de red, inicia la escucha y espera indefinidamente hasta que el proceso termine.

```
def iniciar_servidor(puerto=50051):
    servidor = grpc.server(futures.ThreadPoolExecutor(max_workers
        =10))
    servicio = ServicioArchivos()
    archivos_pb2_grpc.add_ServicioArchivosServicer_to_server(
        servicio, servidor)
    servidor.add_insecure_port(f"[::]:{puerto}")
    servidor.start()
    print("Servidor de archivos activo.")
    print(f"Directorio de trabajo: ./{DIRECTORIO_ALMACEN}")
```

```
    print(f"Escuchando en el puerto {puerto}...")
    servidor.wait_for_termination()
```

5. Lógica del Cliente (`cliente.py`)

El cliente presenta el menú, recopila los datos y construye los mensajes Protocol Buffers correspondientes. El *stub* gRPC convierte esas llamadas en solicitudes remotas y entrega al cliente las respuestas generadas por el servidor.

Mostrar menú: `mostrar_menu()`

Imprime las seis opciones del programa y devuelve como texto la opción ingresada por el usuario. La función no ejecuta operaciones remotas; su responsabilidad es únicamente presentar y recoger la selección.

```
def mostrar_menu():
    print("\n" + "=" * 35)
    print("SISTEMA DE ARCHIVOS REMOTOS")
    print("1. Crear nuevo archivo")
    print("2. Leer un archivo")
    print("3. Agregar contenido a un archivo")
    print("4. Listar archivos disponibles")
    print("5. Eliminar un archivo")
    print("6. Salir")
    print("=" * 35)
    return input("Elige una opción (1-6): ")
```

Iniciar la sesión: `iniciar_cliente()`

La sesión usa por defecto la dirección `localhost:50051`. Abre un canal gRPC sin cifrado, crea el *stub* del servicio y repite el menú hasta que se elija la opción de salida. Cada alternativa crea el tipo de solicitud definido en el protocolo, llama al método remoto correspondiente y muestra la respuesta.

Crear archivo: pide el nombre y contenido inicial, construye un `CrearRequest` y llama a `CrearArchivo`.

```
nombre = input("Ingresa el nombre del archivo (ej. notas.txt): ")
contenido = input("Escribe el contenido inicial: ")
solicitud = archivos_pb2.CrearRequest(
    nombre_archivo=nombre, contenido=contenido)
respuesta = stub.CrearArchivo(solicitud)
print(f"\n[Servidor] -> {respuesta.mensaje}")
```

Leer archivo: envía un `LeerRequest`. Si la respuesta indica éxito, también presenta el contenido recibido.

```
nombre = input("Ingresa el nombre del archivo a leer: ")
solicitud = archivos_pb2.LeerRequest(nombre_archivo=nombre)
respuesta = stub.LeerArchivo(solicitud)
print(f"\n[Servidor] -> {respuesta.mensaje}")
if respuesta.exito:
    print(respuesta.contenido)
```

Agregar contenido: pide el texto nuevo, antepone un salto de línea y lo envía en un `AgregarRequest`. El servidor se encarga de anexarlo al archivo.

```
nombre = input("Ingresa el nombre del archivo a modificar: ")
contenido = input("Escribe el texto a agregar al final: ")
contenido_con_salto = "\n" + contenido
solicitud = archivos_pb2.AgregarRequest(
    nombre_archivo=nombre, contenido=contenido_con_salto)
respuesta = stub.AgregarContenido(solicitud)
print(f"\n[Servidor] -> {respuesta.mensaje}")
```

Listar archivos: envía un `ListarRequest` vacío. Cuando la respuesta es exitosa, recorre el campo `archivos` y muestra cada nombre; si no hay entradas, informa que el directorio está vacío.

```
respuesta = stub.ListarArchivos(archivos_pb2.ListarRequest())
print(f"\n[Servidor] -> {respuesta.mensaje}")
if respuesta.exito:
    if respuesta.archivos:
        for nombre_archivo in respuesta.archivos:
            print(nombre_archivo)
    else:
        print("(El directorio está vacío)")
```

Eliminar archivo: solicita el nombre, lo envía mediante `EliminarRequest` y muestra el mensaje de confirmación o error devuelto por el servidor.

```
nombre = input("Ingresa el nombre del archivo a eliminar: ")
solicitud = archivos_pb2.EliminarRequest(nombre_archivo=nombre)
respuesta = stub.EliminarArchivo(solicitud)
print(f"\n[Servidor] -> {respuesta.mensaje}")
```

Salida y errores de comunicación: la opción 6 termina el ciclo. Si alguna llamada gRPC falla, el bloque `except` captura `grpc.RpcError` y muestra instrucciones para verificar que el servidor siga activo.

```
elif opcion == '6':
    print("\nCerrando conexión. ¡Hasta luego!")
```

```
            break
    else:
        print("Opción no válida. Intenta de nuevo.")

    except grpc.RpcError:
        print("Error de comunicación: No se pudo conectar al servidor.")
        print("Asegúrate de que 'servidor.py' esté en ejecución.")
```

Manual de ejecución

Para ejecutar el sistema, se requieren dos terminales separadas, ambas con el entorno virtual activo.

1. Iniciar el Servidor

El servidor debe levantarse primero para crear el directorio y quedar a la escucha.

`python3 servidor.py`

Nota: Deberá aparecer un mensaje indicando que el directorio fue creado o ya está listo.

2. Iniciar el Cliente

En la segunda terminal, ejecute el cliente:

`python3 cliente.py`

El cliente se conectará a `localhost:50051` por defecto y desplegará el menú de opciones.

Resultados Obtenidos y Pruebas

1. Crear un archivo y comprobar que su contenido queda almacenado.

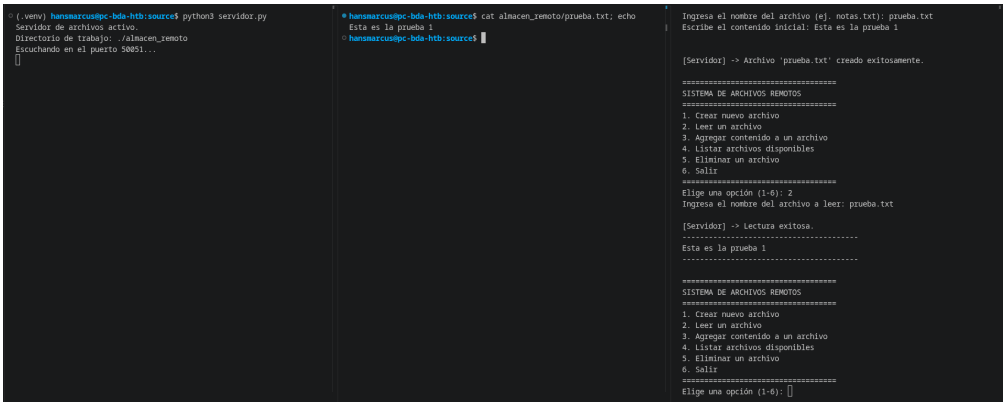


Figure 2: Creación de un archivo y verificación de su contenido.

Como puede observarse, nuestro cliente al conectarse con el servidor de archivos remotos, muestra un menú interactivo que permite al usuario realizar varias operaciones (crear, leer, agregar, listar, eliminar archivos y salir). En la imagen se evidencia que el cliente ha creado un archivo llamado `prueba.txt` con el contenido incial "Esta es la prueba 1", el cual se verifico despues al realizar desde otra terminal el comando `cat almacen_remoto/prueba.txt` y se pudo comprobar que el contenido del archivo es el esperado.

2. Leer un archivo existente y comparar el contenido recibido con el esperado.



Figure 3: Lectura de un archivo existente y verificación de su contenido.

Para esta prueba, en lugar de realizar la lectura con el comando `cat` desde la terminal, se utilizó la opción 2 del menú del cliente para leer el archivo `prueba.txt`. El contenido recibido fue exactamente el mismo que se había escrito previamente, confirmando que la operación de lectura funciona correctamente.

3. Agregar contenido y comprobar que el archivo conserva el texto previo y añade el nuevo.

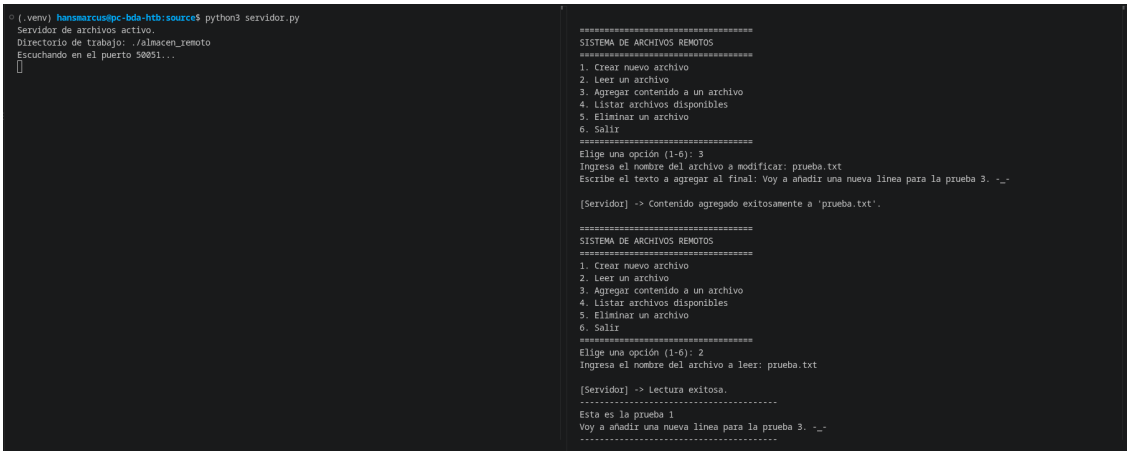


Figure 4: Agregado de contenido a un archivo existente y verificación del resultado.

En esta prueba, se utilizó la opción 3 del menú del cliente para agregar el texto "Voy a añadir una nueva linea para la prueba 3. -_-" al archivo `prueba.txt`. Su contendido fue verificado ahora desde el cliente y se pudo comprobar que el archivo conserva el texto previo y añade el nuevo contenido al final, tal como se esperaba.

4. Solicitar la lectura de un archivo inexistente y verificar el tratamiento del error.

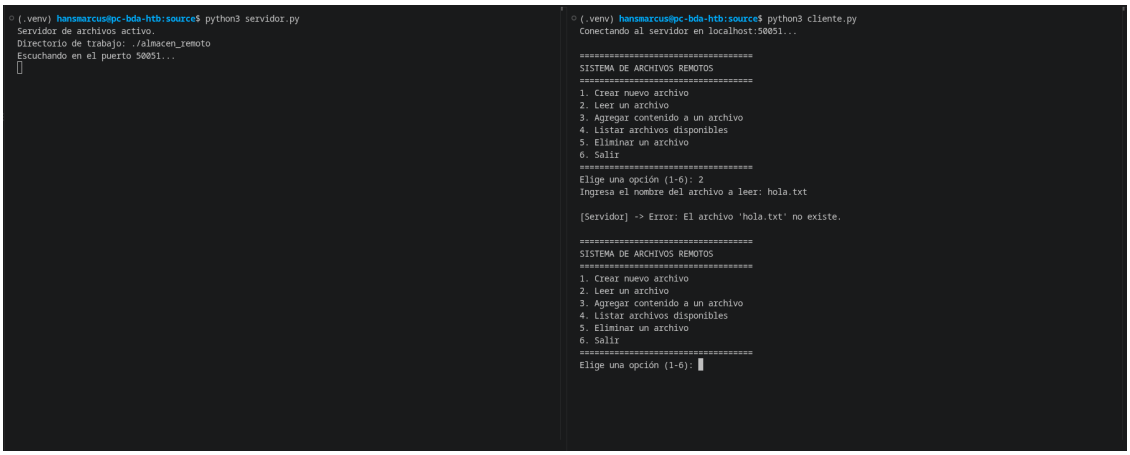


Figure 5: Intento de lectura de un archivo inexistente y manejo del error.

Como se menciono anteriormente, el sistema de archivos remotos implementa un manejo de errores robusto. En esta prueba, se intentó leer un archivo llamado `hola.txt` que no existe en el directorio de almacenamiento. El cliente recibió un mensaje de error claro indicando que el archivo no existe, demostrando que el sistema maneja adecuadamente las solicitudes inválidas.



Figure 6: Intento de escritura en un archivo inexistente y manejo del error.

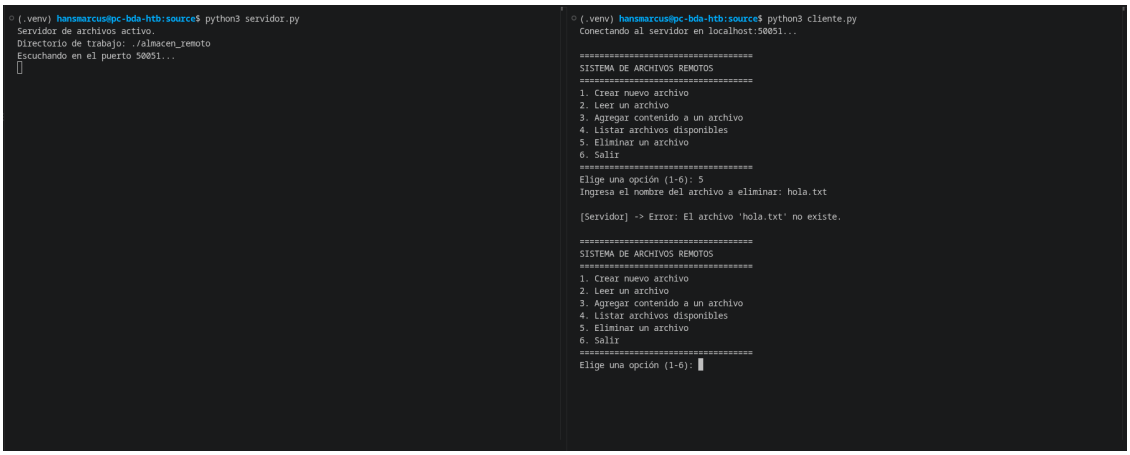


Figure 7: Intento de eliminación de un archivo inexistente y manejo del error.

Adicionalmente, se realizaron pruebas de escritura y eliminación sobre el mismo archivo inexistente. En ambos casos, el servidor respondió con mensajes de error apropiados, confirmando que las operaciones sobre archivos no existentes son correctamente rechazadas.

5. Intentar usar un nombre o una ruta no permitida y verificar que el servidor rechaza la operación.



Figure 8: Intento de crear un archivo con un nombre no permitido y manejo del error.

En esta prueba, se intentó crear un archivo con el nombre `Documents/prueba5.txt`, que contiene una ruta relativa no permitida. El servidor rechazó la operación y devolvió un mensaje de error indicando que el nombre de archivo es inválido, demostrando que la validación de seguridad funciona correctamente y protege contra intentos de escape del directorio confinado.

6. Listar los archivos disponibles y comprobar que no se muestran recursos fuera del directorio autorizado.



Figure 9: Listado de archivos disponibles en el directorio confinado.

El cliente solicitó el listado de archivos disponibles en el directorio confinado. El servidor respondió con éxito, mostrando únicamente los archivos que se encuentran dentro del directorio autorizado (`almacen_remoto`). Ya que nuestra función implementada en el servidor unicamente lista los archivos dentro de este directorio, no se muestran recursos fuera del mismo, garantizando la seguridad y confinamiento del sistema.

7. Detener el servidor y comprobar que el cliente informa que no pudo completar la solicitud.

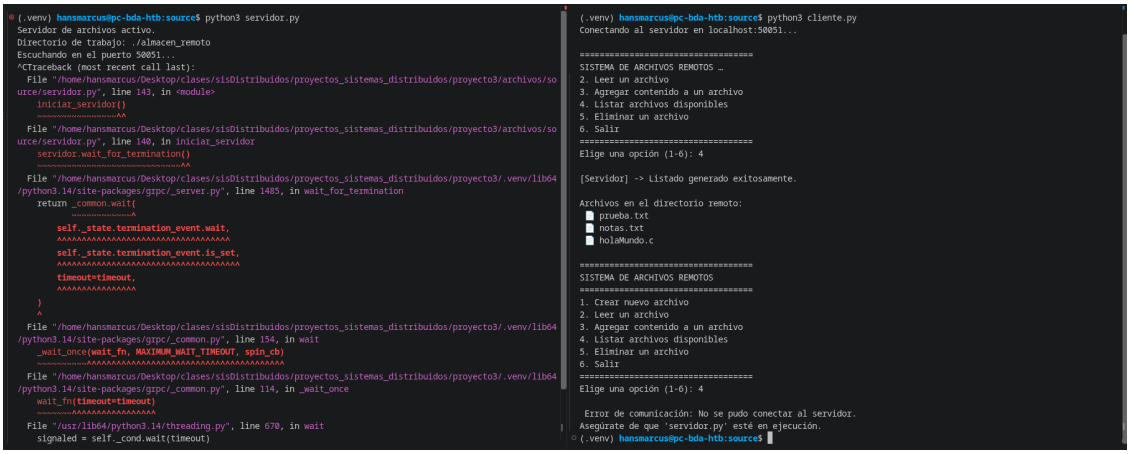


Figure 10: Intento de operación con el servidor detenido y manejo del error de comunicación.

En nuestro caso, el cliente no se deteniene de inmediato cuando el servidor termina su ejecución, sino, que al intentar realizar una operación, el cliente detecta que no puede establecer comunicación con el servidor y muestra un mensaje de error indicando que no se pudo conectar. Esto confirma que el cliente maneja adecuadamente la situación en la que el servidor no está disponible.

8. Matriz de pruebas con entradas, resultados esperados, resultados observados y tratamiento de errores

Escenario de Prueba	Resultado esperado	Resultado Observado	Manejo de errores
Crear un archivo	Archivo creado exitosamente	Archivo creado y contenido verificado	Mensaje de éxito mostrado al usuario
Leer un archivo existente	Contenido del archivo mostrado correctamente	Contenido coincide con lo esperado	Mensaje de éxito mostrado al usuario
Agregar contenido a un archivo existente	Contenido agregado al final del archivo	Contenido previo conservado y nuevo contenido añadido	Mensaje de éxito mostrado al usuario
Leer un archivo inexistente	Mensaje de error indicando que el archivo no existe	Mensaje de error recibido correctamente	Mensaje de error mostrado al usuario
Escribir en un archivo inexistente	Mensaje de error indicando que el archivo no existe	Mensaje de error recibido correctamente	Mensaje de error mostrado al usuario
Eliminar un archivo inexistente	Mensaje de error indicando que el archivo no existe	Mensaje de error recibido correctamente	Mensaje de error mostrado al usuario
Intentar usar un nombre o ruta no permitida	Mensaje de error indicando que el nombre de archivo es inválido	Mensaje de error recibido correctamente	Mensaje de error mostrado al usuario
Listar archivos disponibles	Listado de archivos dentro del directorio confinado	Listado coincide con lo esperado	Mensaje de éxito mostrado al usuario
Detener el servidor y realizar una operación	Mensaje de error indicando que no se pudo conectar al servidor	Mensaje de error recibido correctamente	Mensaje de error mostrado al usuario

Table 1: Matriz de pruebas evaluando conectividad, tolerancia a fallos y contraste de tiempos de ejecución.

Conclusión

El diseño e implementación de este segundo proyecto demostró las poderosas capacidades de gRPC para orquestar un servicio de archivos remotos, sustituyendo las complejas operaciones manuales sobre *sockets* por un modelo estructurado de llamadas a procedimientos remotos. La definición del contrato en Protocol Buffers no solo garantizó la coherencia en la transferencia de datos entre el cliente interactivo y el servidor, sino que también obligó a diseñar una interfaz de errores unificada. Retornar siempre un objeto `Response` con una bandera de éxito y un mensaje legible demostró ser una práctica vital para mantener al cliente informado y evitar rupturas de conexión inesperadas.

El punto más crítico de esta implementación residió en la arquitectura de seguridad y confinamiento perimetral. En sistemas distribuidos reales, donde múltiples clientes pueden acceder a un recurso compartido, confiar a ciegas en el parámetro enviado por el usuario es un fallo crítico. La inclusión del método `_validar_seguridad` ejemplifica cómo se debe blindar un servidor frente a ataques de escalamiento de privilegios o *Path Traversal*. Al interceptar y rechazar proactivamente caracteres de escape jerárquico (como `../` o \), el servidor garantizó que el sistema de archivos del host permaneciera hermético. Esto asegura de manera tajante que ningún cliente, por error o con intenciones maliciosas, pueda listar, sobrescribir o eliminar directorios del sistema operativo o acceder a información confidencial de otros dominios.

En conclusión, el uso conjunto de gRPC para la abstracción de red y Python para las validaciones lógicas locales dio como resultado un sistema robusto. Logró separar exitosamente las responsabilidades: un cliente ligero dedicado exclusivamente a la interacción con el usuario, y un servidor centralizado, defensivo y autónomo, encargado de preservar la integridad y seguridad de la información almacenada.

Ligas al repositorio y video de demostración

- Repositorio del proyecto: https://github.com/lethalSopaper/proyectos_sistemas_distribuidos/tree/main/proyecto3
- Video de demostración: https://drive.google.com/file/d/12etwFrJe3ytfvPabTsVRUk2sHiygWR8_/view?usp=sharing

1 Bibliografía

[1] Sáenz García, E. K. (2026). Notas de clase: Sistemas Distribuidos – Tema 4. Exclusión Mutua